

From Statistical Vulnerability to Cryptographic Security: Evaluating PRNGs via Deep Learning and Algebraic Cryptanalysis

Research Student
Graduate Research Group

June 2026

Abstract

Pseudo-Random Number Generators (PRNGs) form the infrastructural core of modern computational simulations, cryptographic protocols, and numerical modeling. While historical generators exhibit structural biases, modern structural matrix generators and stream ciphers offer varying degrees of architectural resilience. This paper presents a comprehensive empirical evaluation of three prominent PRNG families: the Linear Congruential Generator (LCG), the Mersenne Twister (MT19937), and the ChaCha20 stream cipher-based CSPRNG. We contrast two independent exploitation paradigms: sequence learning via Long Short-Term Memory (LSTM) recurrent networks and deterministic algebraic state recovery. Our empirical and mathematical findings demonstrate that while LSTMs readily exploit the low-order bit periodicities of standard LCG configurations (achieving 100% classification accuracy), they suffer from optimization stagnation when faced with the high-dimensional matrix mixing of MT19937, collapsing directly to the theoretical continuous uniform variance baseline of 0.083333 under trajectory regression. However, we prove that MT19937 remains entirely vulnerable to a deterministic $O(1)$ algebraic untempering algorithm that fully clones the internal 624-element state array from purely observed streams. Finally, we demonstrate that ChaCha20’s non-linear ARX (Add-Rotate-XOR) network successfully resists both deep learning sequence prediction and algebraic matrix inversion. This dichotomy establishes clear boundaries between statistical uniformity and cryptographic security.

1 Introduction

Randomness is an essential operational resource required across modern computer science, powering stochastic Monte Carlo optimizations, physical modeling, statistical sampling, and cryptographic primitives. Because physical computing hardware cannot natively source true thermodynamic entropy at high data throughput, software architectures rely on Pseudo-Random Number Generators (PRNGs). These frameworks accept a compact initial value, designated as the seed (X_0), and apply a determinis-

tic mathematical transform to expand it into an extensive, reproducible stream of values mimicking independent and identically distributed (i.i.d.) variables.

Historically, the quality of random outputs has been evaluated using standard statistical frameworks such as the National Institute of Standards and Technology (NIST) Special Publication 800-22 test battery, the Dieharder suite, and the comprehensive TestU01 suite. These engines verify global statistical properties like equidistribution, block frequency, and empirical uniformity. Concurrently, deep sequence modeling architectures—specifically Long Short-Term Memory (LSTM) networks—have emerged as state-of-the-art instruments for temporal pattern recognition and multi-step time-series forecasting.

However, passing an array of statistical metrics or confusing a neural network does not automatically confer cryptographic security. This study demonstrates a profound theoretical divide: a sequence can achieve exceptional statistical uniformity and cause gradient-based deep learning optimization loops to flatline, while remaining completely vulnerable to low-complexity algebraic reverse engineering. We systematically unpack this phenomenon across three generational milestones of PRNG design: the Linear Congruential Generator (LCG), the Mersenne Twister (MT19937), and the ChaCha20 stream cipher.

2 Architectural Foundations and Mathematical Proofs

2.1 Linear Congruential Generators (LCG)

The classical Linear Congruential Generator produces sequential states using the first-order recurrence relation:

$$X_{n+1} = (a \cdot X_n + c) \pmod{m} \quad (1)$$

where a is the constant multiplier, c is the additive increment, and m is the modulus defining the upper bound of the integer state space. Under production systems like the GNU C Library (`glibc`) runtime implementation of

`rand()`, these parameter fields are hardcoded to the famous constants $a = 1103515245$, $c = 12345$, and $m = 2^{31}$.

A fundamental mathematical flaw within basic LCG architectures concerns the isolation of bit slices, moving from the Most Significant Bit (MSB) to the Least Significant Bit (LSB). When Equation 1 is evaluated over a binary modulus where $m = 2^k$, the sequence of bits at any given position $j < k$ is entirely independent of the values occupying higher-order bit positions. In the extreme case of the absolute LSB ($j = 0$, or evaluating modulo 2), since $a = 1103515245 \equiv 1 \pmod{2}$ and $c = 12345 \equiv 1 \pmod{2}$, the congruence collapses completely to:

$$X_{n+1} \equiv (1 \cdot X_n + 1) \pmod{2} \equiv (X_n + 1) \pmod{2} \quad (2)$$

Thus, if the initial seed state X_0 is odd (1), the next state is guaranteed to be even (0), which in turn produces an odd state (1). This deterministic reduction provides a formal proof that the LSB of a standard LCG is completely non-random, behaving as a trivial alternating clock cycle (0, 1, 0, 1...) with a period length of exactly 2, regardless of the size of the multiplier.

2.2 Mersenne Twister (MT19937) and State Recovery

To overcome the severe bit-level periodicities and small state ceilings of LCGs, the Mersenne Twister (MT19937) shifts the state paradigm from a single scalar register to a large bitwise vector loop spanning a 624-element array of 32-bit words (a total internal state of 19,937 bits). State progression is governed by a matrix linear recurrence relation over the binary finite field $\mathbb{F}_2$:

$$\mathbf{x}_{k+N} = \mathbf{x}_{k+M} \oplus ((\mathbf{x}_k^{\text{upper}} | \mathbf{x}_{k+1}^{\text{lower}}) \mathbf{A}) \quad (3)$$

where $N = 624$, $M = 397$, and $\mathbf{A}$ represents a specialized companion matrix designed to provide a colossal period length of $2^{19937} - 1$. Because new words are generated by linearly mixing bits across indices separated by massive window offsets ($k, k+1, k+397$), the low-order bit periodicities of the LCG are eradicated, ensuring uniform statistical properties across all bit lanes.

However, to improve local equidistribution, every state word $\mathbf{x}$ is passed through a bit-scrambling pipeline called a tempering filter before being emitted to the user as an output word $\mathbf{z}$. The forward tempering function maps $\mathbf{x} \rightarrow \mathbf{z}$ via four sequential steps:

$$y_1 = x \oplus (x \gg 11) \quad (4)$$

$$y_2 = y_1 \oplus ((y_1 \ll 7) \& \mathbf{B}) \quad (5)$$

$$y_3 = y_2 \oplus ((y_2 \ll 15) \& \mathbf{C}) \quad (6)$$

$$z = y_3 \oplus (y_3 \gg 18) \quad (7)$$

where $\mathbf{B} = 0x9D2C5680$ and $\mathbf{C} = 0xEFC60000$ are constant hexadecimal bitmasks.

Because every single component in this tempering operation is strictly linear over $\mathbb{F}_2$, the entire transformation is

bi-directional and perfectly invertible. Reversing the operations requires working backward from the output z to the state x using bitwise algebra:

1. **Inverting Step 4:** $z = y_3 \oplus (y_3 \gg 18)$. Because a right-shift by 18 moves bits out of the top half of a 32-bit register, the upper 18 bits of z are completely unaffected by the XOR operation. Therefore, we can immediately state that $y_3^{\text{upper}} = z^{\text{upper}}$. Shifting these known bits right by 18 and XORing them back into the lower section of z fully recovers the rest of the word. Thus, the exact algebraic inverse is:

$$y_3 = z \oplus (z \gg 18) \quad (8)$$

2. **Inverting Step 3:** $y_3 = y_2 \oplus ((y_2 \ll 15) \& \mathbf{C})$. A left-shift by 15 means the bottom 15 bits are filled with zeros. Consequently, the lower 15 bits of y_3 are pristine copies of y_2 . We can extract these lower 15 bits, shift them left by 15, mask them with $\mathbf{C}$, and XOR them with y_3 to reconstruct the upper half of y_2 .
3. **Inverting Step 2:** $y_2 = y_1 \oplus ((y_1 \ll 7) \& \mathbf{B})$. This follows the exact same logic as Step 3, executed iteratively in blocks of 7 bits from right to left (LSB to MSB) to resolve the cascading shifts.
4. **Inverting Step 1:** $y_1 = x \oplus (x \gg 11)$. This follows the exact same logic as Step 4, executed iteratively in blocks of 11 bits from left to right (MSB to LSB).

This formal mathematical tracking confirms that the entire output mechanism is transparent. An adversary who records exactly 624 consecutive outputs can run them through these inverse formulas to reconstruct the internal 624-word state array, bypassing all seed security with $O(1)$ computational effort.

2.3 ChaCha20 Cryptographic Stream Cipher

To bridge the gap between statistical complexity and real-world security, a Cryptographically Secure PRNG (CSPRNG) must satisfy the Next-Bit Test, meaning that no polynomial-time algorithm given k bits of history can guess the $(k+1)$ -th bit with a probability higher than 50%. The modern standard for achieving this is the ChaCha20 stream cipher.

ChaCha20 manages a 512-bit internal block state configured as a 4×4 matrix of 32-bit words, partitioned into four constant words ("expand 32-byte k"), eight secret key words (the initialization seed), one block counter word, and three unique nonce words. Random output blocks are generated by running this matrix through 20 rounds of the symmetric Quarter-Round (QR) function, which al-

ters four active indices (a, b, c, d) at a time:

$$a = a + b; \quad d = (d \oplus a) \lll 16; \quad (9)$$

$$c = c + d; \quad b = (b \oplus c) \lll 12; \quad (10)$$

$$a = a + b; \quad d = (d \oplus a) \lll 8; \quad (11)$$

$$c = c + d; \quad b = (b \oplus c) \lll 7; \quad (12)$$

where $+$ represents modular addition over 2^{32} , $\oplus$ is a bit-wise XOR, and $\lll$ represents a bitwise left-rotation.

The critical security advantage of ChaCha20 over MT19937 lies in its **ARX (Add-Rotate-XOR)** architecture. While bitwise XOR and rotations are linear transformations over $\mathbb{F}_2$, **modular addition (+) is highly non-linear** due to the creation of arithmetic carry bits that propagate from right to left across bit lanes. Interleaving addition with XOR breaks the algebraic linearity of the system, causing a single bit change to avalanche across the entire 512-bit block after 20 rounds. This prevents any form of matrix inversion or linear state recovery.

3 Methodological Framework

We implement two distinct evaluation frameworks to empirically test the security limits of these generators.

3.1 Sequence Prediction via Deep LSTM Networks

We frame the machine learning verification suite across two distinct optimization tasks using a 64-unit Recurrent LSTM architecture over a sliding history window ($T = 50$):

3.1.1 Bitwise Stream Classification

The input sequence is converted into a continuous stream of isolated bit indexes (e.g., extracting purely the LSB or purely the MSB). The network is trained as a binary classifier to predict whether the next trailing bit is a 0 or a 1. The system uses a final Sigmoid layer to output a probability $\hat{y}_t \in [0, 1]$ and is optimized using Binary Cross-Entropy (BCE) loss:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (13)$$

A generator achieves perfect statistical security against this model if the classification accuracy settles exactly at 50.00%, indicating that the sequence is indistinguishable from flipping a fair coin.

3.1.2 Numerical Trajectory Regression

To test if a neural network can track full numerical outputs directly rather than isolated bits, we must bypass the traditional classification layer. Trying to classify integers over a massive 32-bit state space (2^{32} categories) is computationally impossible. Instead, we convert the task into a

regression problem. We first normalize the raw generated integers into continuous floating-point scalars:

$$\bar{X}_n = \frac{X_n}{\text{Modulus } m} \in [0, 1] \quad (14)$$

The LSTM’s output layer is stripped of all non-linear activation functions to emit a raw continuous value $\hat{y}_t$, and the parameters are updated via Mean Squared Error (MSE) loss:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (15)$$

To establish a baseline for true randomness under this regression framework, we evaluate the mathematical variance (σ^2) of a perfectly uniform, continuous random distribution bounded tightly between $[0, 1]$. Using continuous probability density integration, we find:

$$\sigma^2 = \int_0^1 (x - \mu)^2 \cdot 1 \, dx = \frac{(b - a)^2}{12} = \frac{(1 - 0)^2}{12} \approx 0.083333 \quad (16)$$

If a sequence contains zero learnable trajectories or linear gradients, the optimal mathematical strategy for the neural network to minimize Equation 15 is to abandon sequence dependencies and consistently predict the global mean ($\mu = 0.5$). Doing so forces the empirical validation loss to converge precisely to the distribution’s variance, 0.083333. Any empirical loss that drops significantly below this threshold proves that the LSTM has successfully discovered a sequential trajectory pattern.

3.2 Algebraic Matrix Inversion Execution

To contrast the gradient descent behavior of the LSTM, we implement a production-grade algebraic cryptanalysis framework designed to exploit the linear vulnerabilities of the Mersenne Twister. The script intercepts a stream of 624 raw 32-bit words, applies bitwise masking and shifting sequences to unroll the forward tempering steps (4-7), maps the unshifted words back to a sequential vector array, and injects this array directly into a native C-level state tuple `(3, state_array + [624], None)` to verify 100% forward predictability.

4 Empirical Results and Observations

The evaluation trials across the different data representations and generators produced distinct behavioral transitions, summarized in Table 1.

4.1 The Normalization and Mismatch Classification Trap

During initial experimental setups, a significant machine learning trap was uncovered. Piping large raw integers directly into a standard classification model compiled with

Table 1: Predictive and Algebraic Performance Metrics Across Randomness Sources

PRNG Family / Target Domain	LSTM Sequence Metric	Algebraic Inversion
LCG (glibc Core)		
Least Significant Bit (LSB)	100.00% Classification	Trivial Linear Cycle
Most Significant Bit (MSB)	50.01% Classification	Non-Linear Overflow Cascades
Normalized Floating-Points	0.083321 MSE (Stagnation)	Structural Scale Blindness
MT19937		
Least Significant Bit (LSB)	50.03% Classification	100.00% State Recovery
Most Significant Bit (MSB)	49.97% Classification	100.00% State Recovery
Normalized Floating-Points	0.083512 MSE (Stagnation)	100.00% State Recovery
ChaCha20 Stream Cipher		
Isolated Bit Lanes (Any)	50.00% Classification	$O(2^{256})$ Cryptographic Wall
Normalized Floating-Points	0.083331 MSE (Stagnation)	$O(2^{256})$ Cryptographic Wall

a binary sigmoid output layer forced the network’s predictions to round to 0 or 1. Because a rounded classification prediction can never match a multi-million magnitude integer label, the tracking engine recorded a deceptive accuracy score of exactly **0.00%**. Resolving this by switching to normalized values and linear regression output states revealed the true behavioral trends of the sequence models.

4.2 Analysis of LCG Behavior

When trained on the LSB stream of the LCG, the LSTM classifier achieved a flawless **100.00% accuracy** within the first training epoch. This empirical finding matches our mathematical proof: the LSB is trapped in a simple alternating pattern that the recurrent cell weights map instantly.

However, when evaluating the full normalized numbers directly via regression, the LSTM achieved an MSE of **0.083321**, falling right on top of our random variance baseline (16). This exposes a vulnerability known as **scale blindness**. Because the wide variance of the large floating-point numbers overwhelms the network’s optimization gradient, the model fails to see that its lowest binary sub-component is completely broken.

4.3 Analysis of MT19937 Behavior: The Illusion of Randomness

The Mersenne Twister completely neutralized the deep learning models. The classification accuracy on both the LSB and MSB dropped to **50.03%**, and the trajectory regression loss leveled out at **0.083512 MSE**. Because the bits are thoroughly mixed across a 624-step structural index vector loop, the temporal memory of the LSTM cell cannot track the high-dimensional linear equations, causing its gradient paths to flatline and always predict the mid-point average of 0.5.

However, our deterministic algebraic cryptanalysis framework bypassed gradient descent entirely. By running

the unrolled bitwise untempering algorithm, the engine inverted the bitmasks and shifts in less than 1 millisecond, fully recovering the internal matrix array. This allowed us to clone the generator state and predict every future number with **100.00% accuracy**, demonstrating that the LSTM’s failure to find patterns was merely an illusion of security.

4.4 Analysis of ChaCha20 Cryptographic Integrity

ChaCha20 successfully resisted both exploration methodologies. The LSTM network was restricted to an exact baseline score of **50.00% accuracy** and **0.083331 MSE**, behaving identically to pure thermodynamic noise. Concurrently, the non-linear addition operations inside the ARX Quarter-Rounds blocked the linear Gaussian elimination used to break the Mersenne Twister. State recovery remains bound to the absolute cryptographic complexity ceiling of $O(2^{256})$ key operations.

5 Conclusion

This study establishes clear architectural boundaries when using sequence-learning neural networks to audit random number generators. LSTMs excel at identifying simple, low-dimensional algebraic defects, such as LCG bit lanes, but suffer from severe scale blindness when analyzing full integer values directly. Furthermore, the complete breakdown of the Mersenne Twister against our $O(1)$ algebraic untempering engine proves that deep learning models are blind to high-dimensional linear structures. True cryptographic security requires non-linear mathematical primitives—such as the ARX structure of ChaCha20—to eliminate state recovery vulnerabilities.

References

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [2] M. Matsumoto and T. Nishimura, “Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator,” *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, vol. 8, no. 1, pp. 3–30, 1998.
- [3] D. J. Bernstein, “ChaCha, a variant of Salsa20,” *Workshop Record of SASC*, 2008.
- [4] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [6] L. E. Bassham III, A. L. Rukhin, J. Soto, et al., “A statistical test suite for random and pseudorandom number generators for cryptographic applications,” *NIST Special Publication 800-22 Revision 1a*, 2010.
- [7] L. Pasqualini and M. Parton, “Pseudo Random Number Generation: a Reinforcement Learning approach,” *Procedia Computer Science*, vol. 170, pp. 1122–1127, 2020.